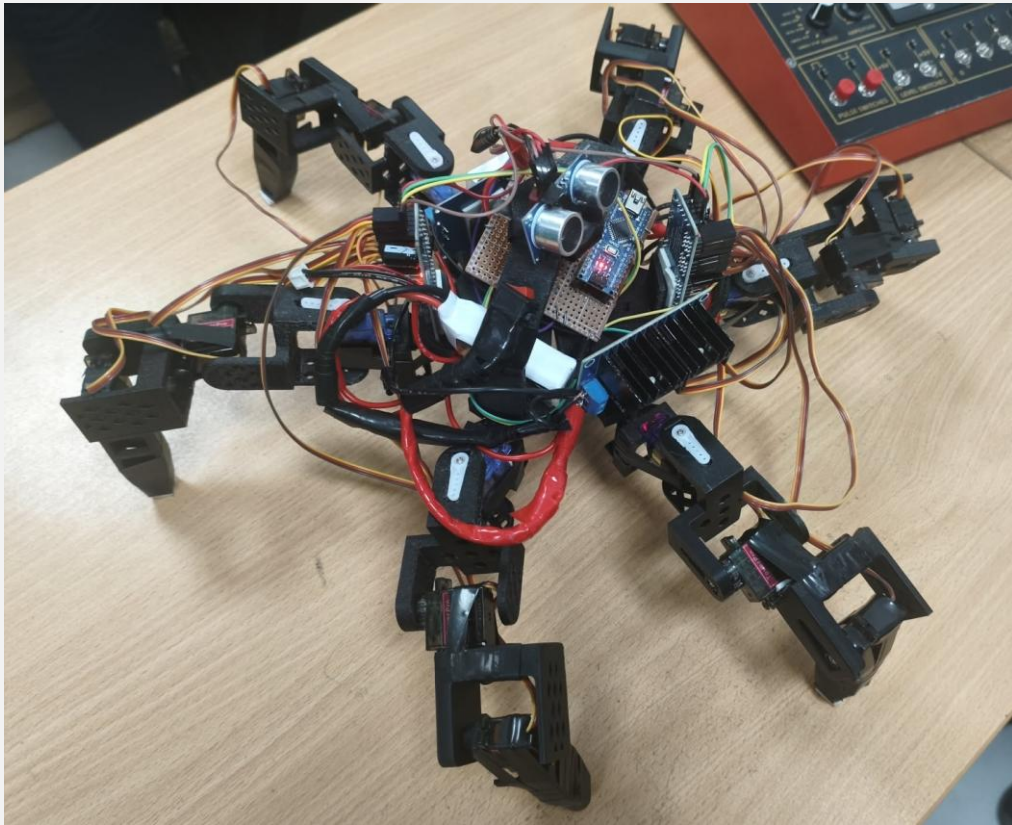


HexEcho

Complete Build Manual

Mechanical Design • Electronics • Firmware • Assembly • Calibration



v1.0 — 2026

Table of Contents

1. Overview
2. Bill of Materials
3. Tools Required
4. Safety Notes
5. Mechanical Design
6. Electronics Design
7. Firmware
8. Assembly Instructions
9. Exploded View & Final Assembly
10. Testing & Calibration
11. Troubleshooting
12. Roadmap
13. Appendix A — Full Firmware Source
14. Appendix B — Credits, License & Revision History

1. Overview

HexEcho is an 18 degree-of-freedom autonomous hexapod robot. It uses a wave gait (one leg moves at a time) for maximum stability, an HC-SR04 ultrasonic sensor for obstacle detection, and a dual PCA9685 servo driver architecture controlled by an Arduino Nano. The entire chassis is 3D printed in PETG.

This manual documents the complete build — mechanical parts, wiring, firmware, and assembly — in enough detail that the robot can be reproduced from these files.

Top-Level Layout

Leg numbering and orientation used throughout this manual and in the firmware:

```
[ FRONT ]
L0       R3
L1       R4
L2       R5
[ BACK ]

L = left side (PCA9685 #1, address 0x40)
R = right side (PCA9685 #2, address 0x41)
```

2. Bill of Materials

Component	Specification	Qty
Microcontroller	Arduino Nano (ATmega328P)	1
Servo Driver	PCA9685 16-channel PWM driver	2
Servo (Femur/Tibia)	MG90S metal gear micro servo	12
Servo (Coxa)	SG90 micro servo	6
Ultrasonic Sensor	HC-SR04	1
Battery	2S LiPo 7.4V 1000mAh, T-plug	1
Balance Charger	B3 2S/3S LiPo balance charger	1
Voltage Monitor	1S–8S cell voltage checker/alarm	1
Buck Converter (servo rail)	XH-M404, 5A cont. / 8A peak, adjustable	1
Buck Converter (logic rail)	LM2596 adjustable, 5V output	1
Filter Capacitor	1000µF 16V electrolytic	2
Power Switch	Rocker or slide switch	1
T-Plug Connector Pair	Male/female Deans-style	1 pair
Wire — power	22AWG stranded hookup wire (red/black)	~2m
Wire — signal	Jumper wires (M-M, M-F, F-F)	1 kit

Perfboard	≥ 70×90mm	1
M2 Heat Set Insert	Brass, knurled, for M2 screws	~30
M3 Heat Set Insert	Brass, knurled, for M3 screws	~30
M2 Machine Screws	6–10mm length, carbon steel	~40
M3 Machine Screws	8–12mm length, carbon steel	~20
Chassis Parts	Custom PETG 3D printed (see Section 5)	1 set (20 parts)

3. Tools Required

- Soldering iron (40–60W) with stand, solder (60/40 or 63/37 rosin core), flux paste
- Precision screwdriver set (PH00 / PH0 / PH1)
- Small wire cutters / strippers
- Digital multimeter
- IPA (isopropyl alcohol, 70%+) for cleaning contacts
- Computer with Arduino IDE installed
- USB cable (data-capable, matching your Nano's port type)

4. Safety Notes

BATTERY SAFETY: Always charge the LiPo battery with a balance charger (B3 or similar). Never charge an unattended battery. Never discharge below 3.0V per cell (set your voltage alarm to trigger at 3.5V/cell for a safety margin). Disconnect the battery and the voltage checker after every session — both draw a small current even when the robot is off.

ASSEMBLY: Heat set inserts require the soldering iron at ~200–220°C for PETG. Thread a sacrificial screw into the insert before installing to keep the threads clean.

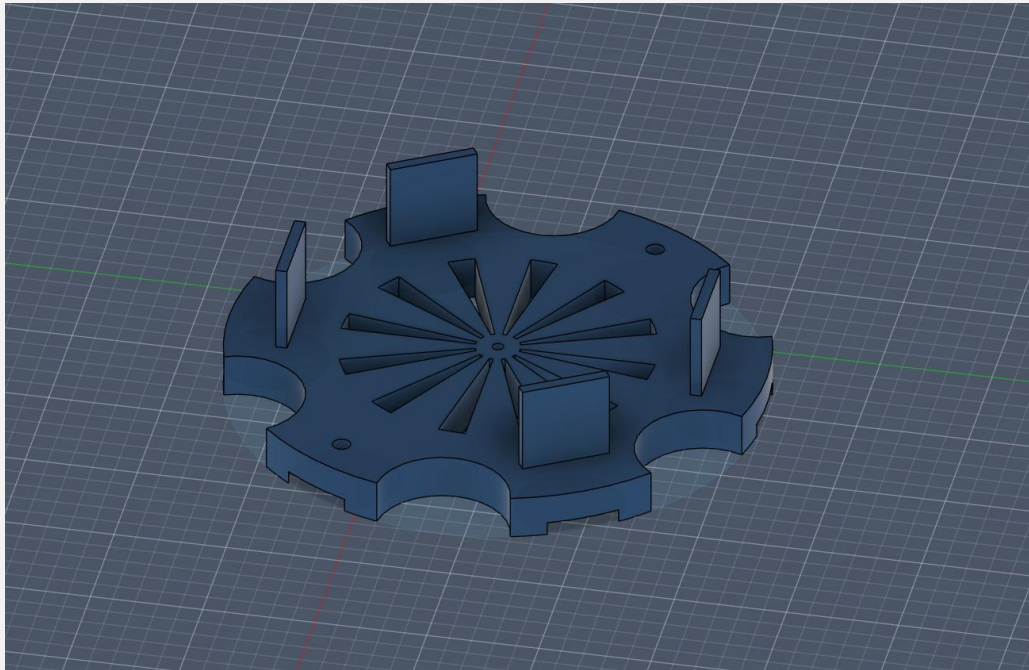
5. Mechanical Design

All structural parts are modelled in Autodesk Fusion 360 and 3D printed in PETG. Final leg segment lengths used for inverse kinematics reference are listed below (measured from the assembled physical legs — confirm against your own build before tuning IK).

Segment	Length (mm)	Notes
Coxa	60.0	Body joint to femur joint, includes bracket + horn
Femur	50.0	Femur joint to tibia joint, includes servo body
Tibia	70.0	Tibia joint to foot tip

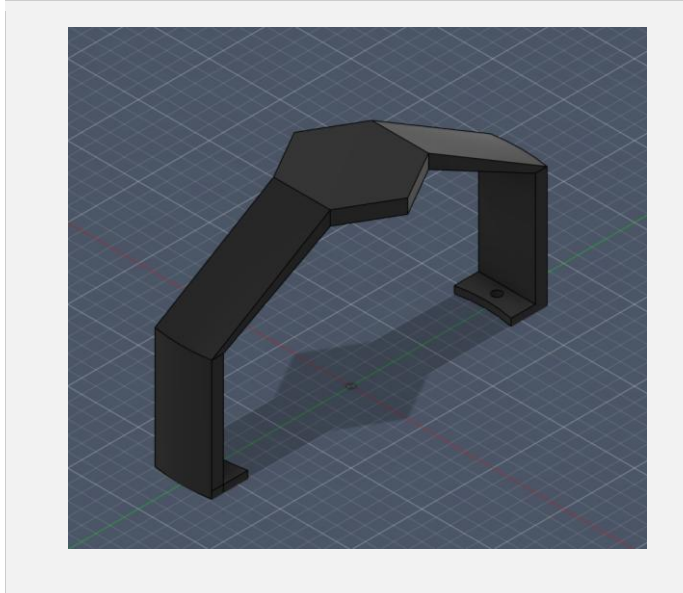
5.1 Body Cylinder

Houses the battery and all the electronics. Outer walls are used to mount pca9685 and two buck converters.



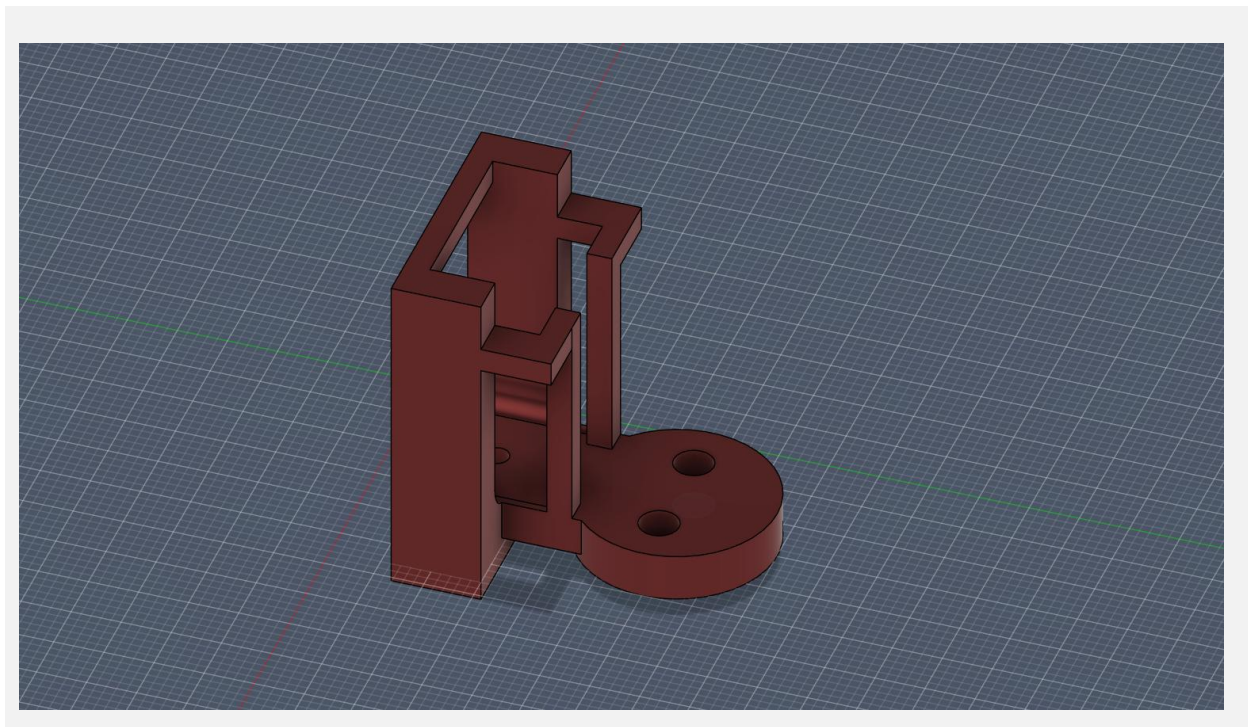
5.2 Cover Plate

Mounts to the body cylinder. Carries the ultrasonic sensor HC-SR04.



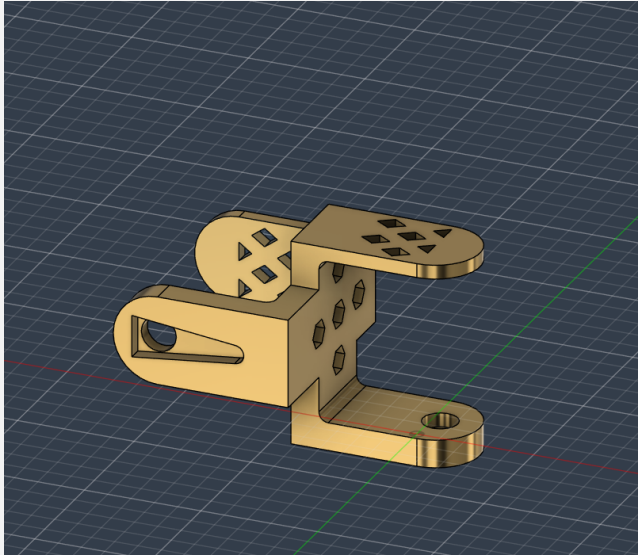
5.4 Body Hook

Connects the body with the coxa bracket.



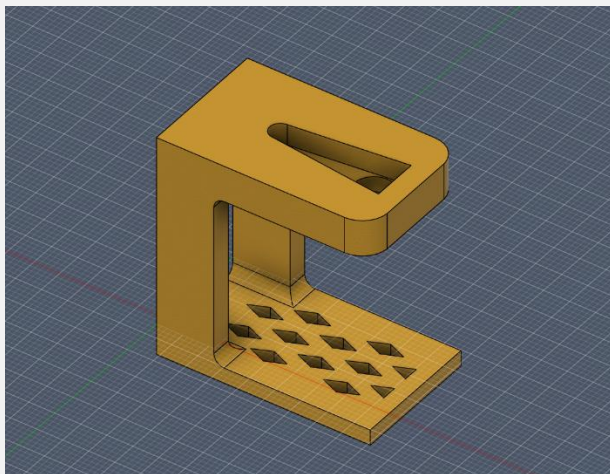
5.4 Coxa Bracket

The most structurally complex part — mounts the coxa servo and connects the leg to the body hook. Walls increased to 6mm with R2–R5mm fillets at all thickness transitions after an early print failed by delaminating along the Z-axis at a thin-wall/thick-wall junction.



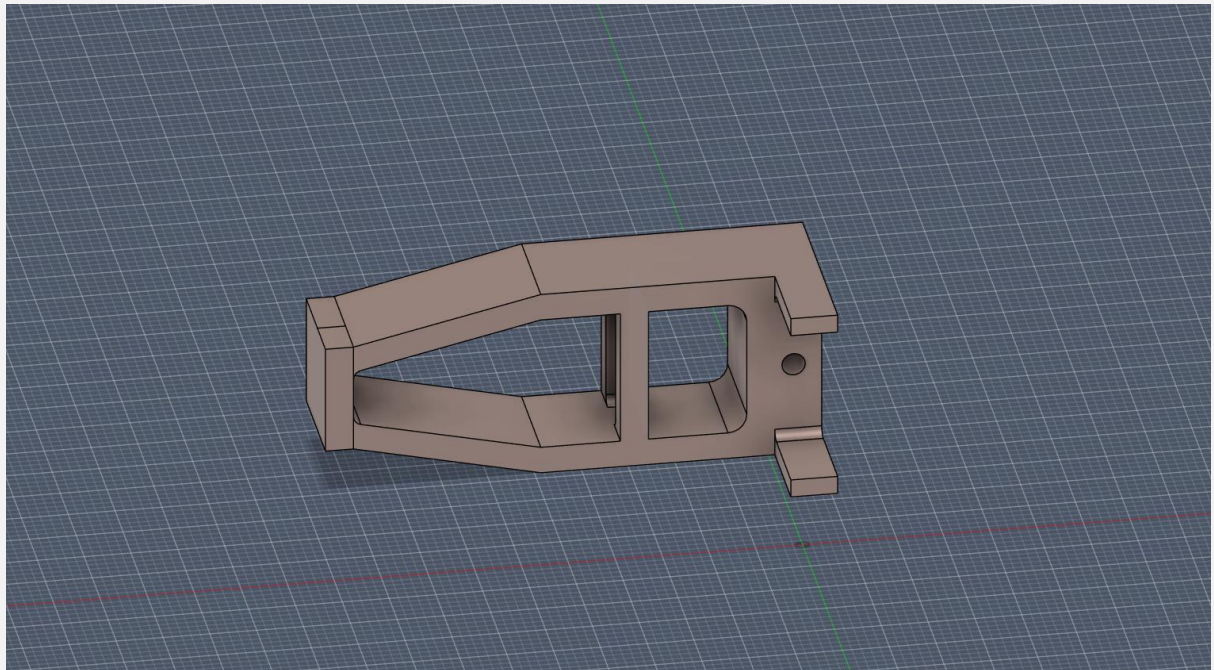
5.5 Femur Segment

Houses the femur servo. Includes a cross-shaped horn socket (press fit, ~23mm across, 3.5mm deep) plus a single M2 retention screw through the horn center.



5.6 Tibia Segment

Passive segment, no servo. Tapered profile (wider at femur end, narrower at foot). Foot tip uses folded double-sided foam tape in place of a dedicated TPU foot cap for grip on smooth surfaces.



5.7 Fastener & Insert Reference

Use Case	Hardware	Hole Diameter	Depth
Cover → body	M2 + heat set insert	2.5mm	4mm
Servo body → bracket/segment	M2, direct into plastic	2.5mm (clearance) / 2.2mm (self-tap)	—
Leg → leg / leg → body structural joint	M3 + heat set insert	4mm	6mm
M3 clearance (pass-through)	M3 screw shaft	3.2–3.5mm	through

5.8 Print Settings

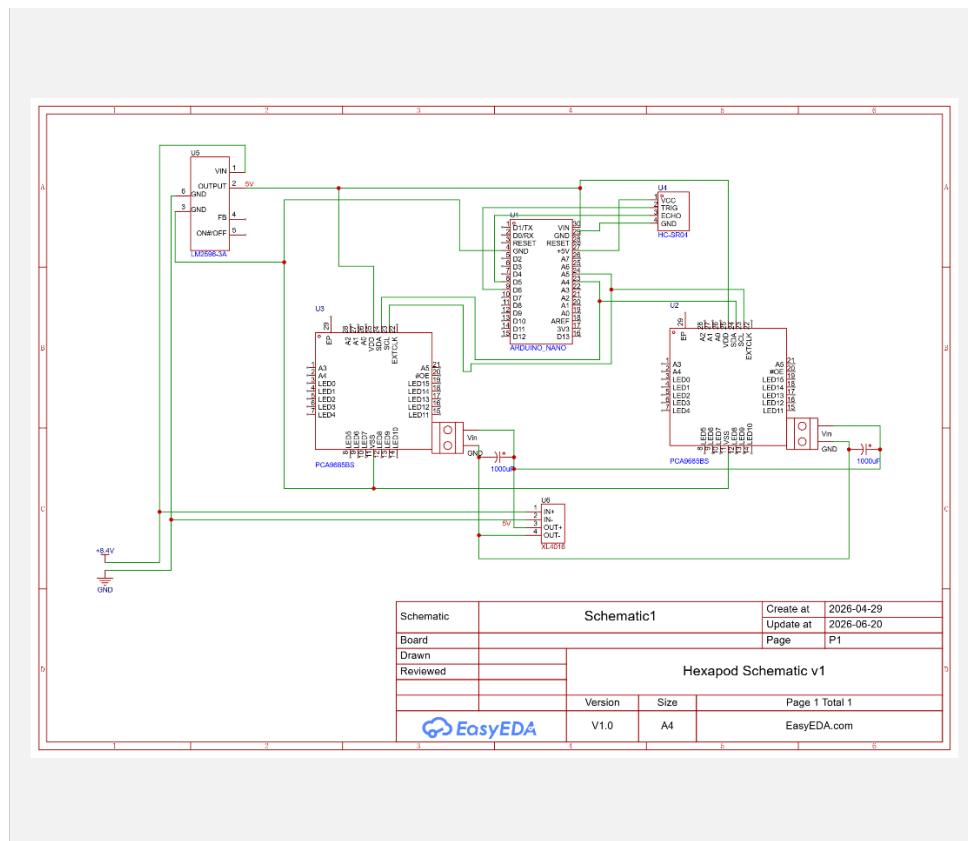
Part	Infill	Walls	Layer Height	Orientation
Body cylinder	25–35%	3	0.2mm	Open face down
Cover plate	25–35%	3	0.2mm	Flat
Body Hook	35–40%	3	0.2mm	Cylinders up
Coxa bracket	35–40%	4–6 (6mm wall)	0.2mm	Broken wall parallel to bed
Femur	35–40%	3 (3mm wall)	0.2mm	Longest face flat
Tibia	35–40%	3 (3mm wall)	0.2mm	Longest face flat

Material: PETG throughout. Nozzle: 0.4mm. Gyroid infill pattern preferred for fatigue resistance.

6. Electronics Design

6.1 Full Circuit Schematic

Complete schematic drawn in EasyEDA, covering all connections listed above plus per-channel servo wiring.



6.2 Power Architecture

Two independent regulated rails from a single 2S LiPo, to isolate servo current spikes from the microcontroller's logic supply:

- Servo rail — XH-M404 buck converter, set to 5V, feeds both PCA9685 V+ screw terminals. One 1000 μ F 16V capacitor across V+/GND at each PCA9685 for local spike filtering.
- Logic rail — LM2596 buck converter, set to 5V, feeds Arduino Nano VIN. Nano's own 5V pin then powers both PCA9685 VCC (logic) pins and the HC-SR04.

6.3 Microcontroller & Servo Driver Wiring

From	To	Purpose
Nano A4 (SDA)	PCA9685 #1 + #2 SDA	I2C data, shared bus
Nano A5 (SCL)	PCA9685 #1 + #2 SCL	I2C clock, shared bus
Nano 5V	PCA9685 #1 + #2 VCC	Logic power
Nano GND	Common ground rail	Logic ground
XH-M404 output +	PCA9685 #1 + #2 V+ (screw terminal)	Servo power
XH-M404 output -	Common ground rail	Servo ground
LM2596 output +	Nano VIN	Logic supply

I2C ADDRESSING: PCA9685 #1 is address 0x40 (left legs 0,1,2). PCA9685 #2 has its A0 jumper pad bridged to set address 0x41 (right legs 3,4,5).

6.4 Sensor Wiring

HC-SR04 Pin	Connects To
VCC	Nano 5V
GND	Common ground rail
TRIG	Nano D6
ECHO	Nano D5

7. Firmware

7.1 Software Architecture

The firmware is built around three core mechanisms:

1. Interpolation engine — every servo move is broken into 1° ticks (12ms apart by default) so all 18 servos glide smoothly and simultaneously rather than snapping.
2. Hold delay — replaces plain delay() calls. Continuously re-sends the current PWM value to every servo during any pause, preventing servos from going limp and working around occasional state loss on clone PCA9685 boards.
3. Per-leg state struct — each leg tracks its own current angle, target angle, PCA9685 board/channel assignment, and whether its coxa direction needs mirroring (right-side legs).

7.2 Channel Map

Leg	Side	Board	Coxa Ch	Femur Ch	Tibia Ch
0	Left — Front	0x40	0	1	2
1	Left — Mid	0x40	3	4	5
2	Left — Back	0x40	6	7	8
3	Right — Front	0x41	0	1	2
4	Right — Mid	0x41	3	4	5
5	Right — Back	0x41	6	7	8

7.3 Key Parameters Reference

Parameter	Default	Description
COXA_HOME_DEG	90°	Coxa angle when leg is centered
FEMUR_HOME_DEG	60°	Femur angle in standing pose
TIBIA_HOME_DEG	110°	Tibia angle for flat foot contact
STEP_COXA_SWING	25°	Coxa travel per forward/back step
LIFT_FEMUR_DEG	20°	Femur rise during swing phase
TURN_COXA_OFFSET	22°	Extra coxa offset applied when turning
INTERP_STEP_DEG / INTERP_TICK_MS	1° / 12ms	Interpolation smoothness/speed
PHASE_SETTLE_MS	120ms	Pause after each gait phase
STEP_SETTLE_MS	300ms	Pause after a leg finishes planting
CYCLE_SETTLE_MS	700ms	Pause after a full 6-leg cycle
OBSTACLE_CM	20cm	Distance threshold for obstacle avoidance
PWM_MIN / PWM_MAX	150 / 600	PCA9685 pulse counts for 0°/180° — tune per servo batch

7.4 Wave Gait Logic

Wave order alternates sides to keep the support polygon balanced across the remaining five stance legs at all times:

```
WAVE_ORDER = { 0, 3, 1, 4, 2, 5 }  
( L-Front → R-Front → L-Mid → R-Mid → L-Back → R-Back )
```

Each leg step has three phases: Lift (femur rises, coxa stays centered) → Swing (coxa rotates toward target with femur still lifted) → Plant (femur lowers, foot returns to ground).

7.5 Obstacle Avoidance Logic

4. Read HC-SR04. If clear ($\geq$ OBSTACLE_CM) — step forward.
5. If blocked — turn left one cycle, re-check.
6. If still blocked — turn right twice (undo the left turn, then face right), re-check.
7. If still blocked — turn right once more (now facing roughly opposite original heading), re-check.
8. If still blocked in all directions — return to home position and halt, holding all servos at home indefinitely.

7.6 Calibration Procedure

9. Upload firmware with all home angles at default (90°/60°/110°). Power on without legs attached.
10. Verify each servo reaches a sensible angle. If a servo overshoots or undershoots, adjust PWM_MIN / PWM_MAX for that servo's batch.
11. Attach legs one at a time. Confirm the robot stands stably on all 6 legs at home position before enabling gait.
12. Adjust FEMUR_HOME_DEG / TIBIA_HOME_DEG in small increments until the robot stands level with all feet flat on the ground.
13. Test waveGait(+1) alone (call from setup() temporarily) and observe one full cycle. Adjust STEP_COXA_SWING and LIFT_FEMUR_DEG if the foot drags or overshoots.
14. Re-enable obstacleAvoidanceLoop() in the main loop once gait looks stable.

8. Assembly Instructions

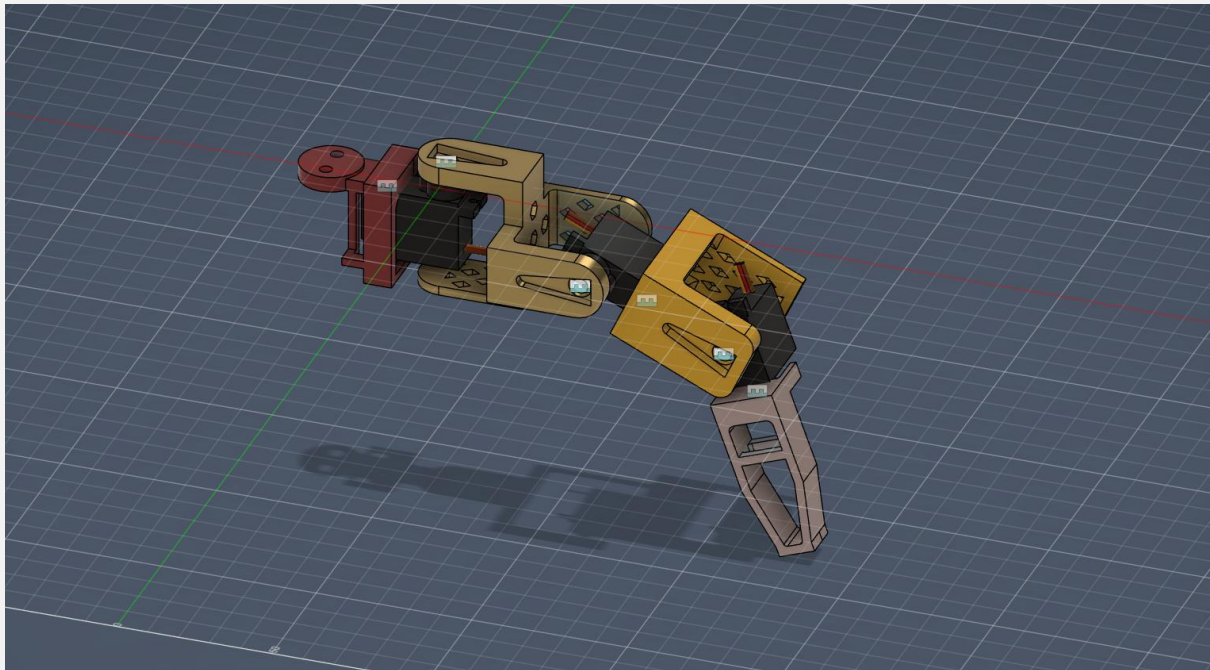
8.1 Heat Set Insert Installation

15. Thread a sacrificial M2/M3 screw into the insert to protect the threads.
16. Heat soldering iron to 200–220°C.

17. Place insert on the corresponding hole, press iron tip gently on top of the insert.
18. Apply light downward pressure as the insert sinks flush with the surface.
19. Remove iron, hold still for ~15 seconds to cool, then remove the sacrificial screw.

8.2 Leg Sub-Assembly

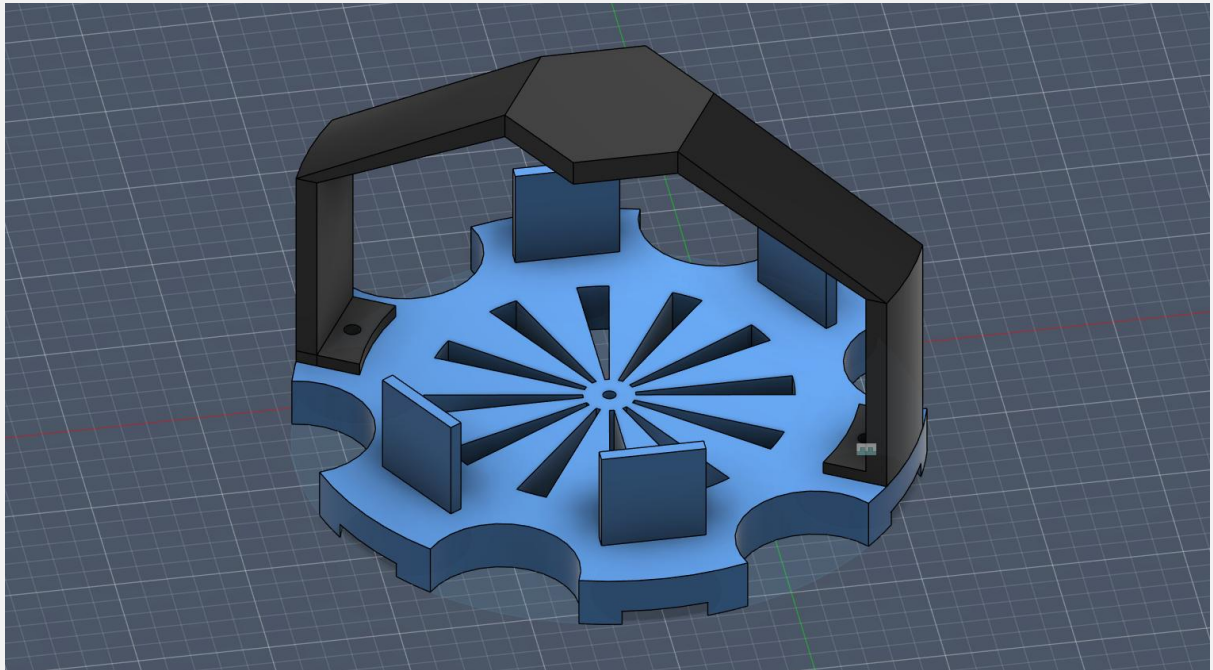
20. Mount SG90 into coxa bracket with M2 screws through the flange.
21. Mount MG90S into femur segment with M2 screws through the flange.
22. Mount MG90S into tibia mounting point on femur (tibia itself carries no servo).
23. Press-fit each servo horn into its corresponding cross socket, secure with a single M2 screw through the horn center.
24. Join coxa → femur and femur → tibia with M2 screws into heat set inserts (2 screws per joint, ~16–18mm apart).



8.3 Body & Cover Assembly

25. Install M3 heat set inserts at all 6 leg mounting points on the body cylinder.
26. Attach all 6 leg assemblies to the body using M3 screws (2 per leg).
27. Insert battery into body cylinder, route T-plug and balance lead out through the wiring slot.
28. Attach cover plate over body with M2 heat set inserts and screws.

29. Attach cover plate to body.



8.4 Electronics Mounting

- 30. Mount Arduino Nano on the body cylinder using M2 screws or double sided foam tape.
- 31. Mount both PCA9685 boards using double-sided foam tape with the outer walls.
- 32. Mount both buck converters using double-sided foam tape.
- 33. Mount HC-SR04 on the cover plate, elevated above leg sweep height.

8.5 Wiring Sequence

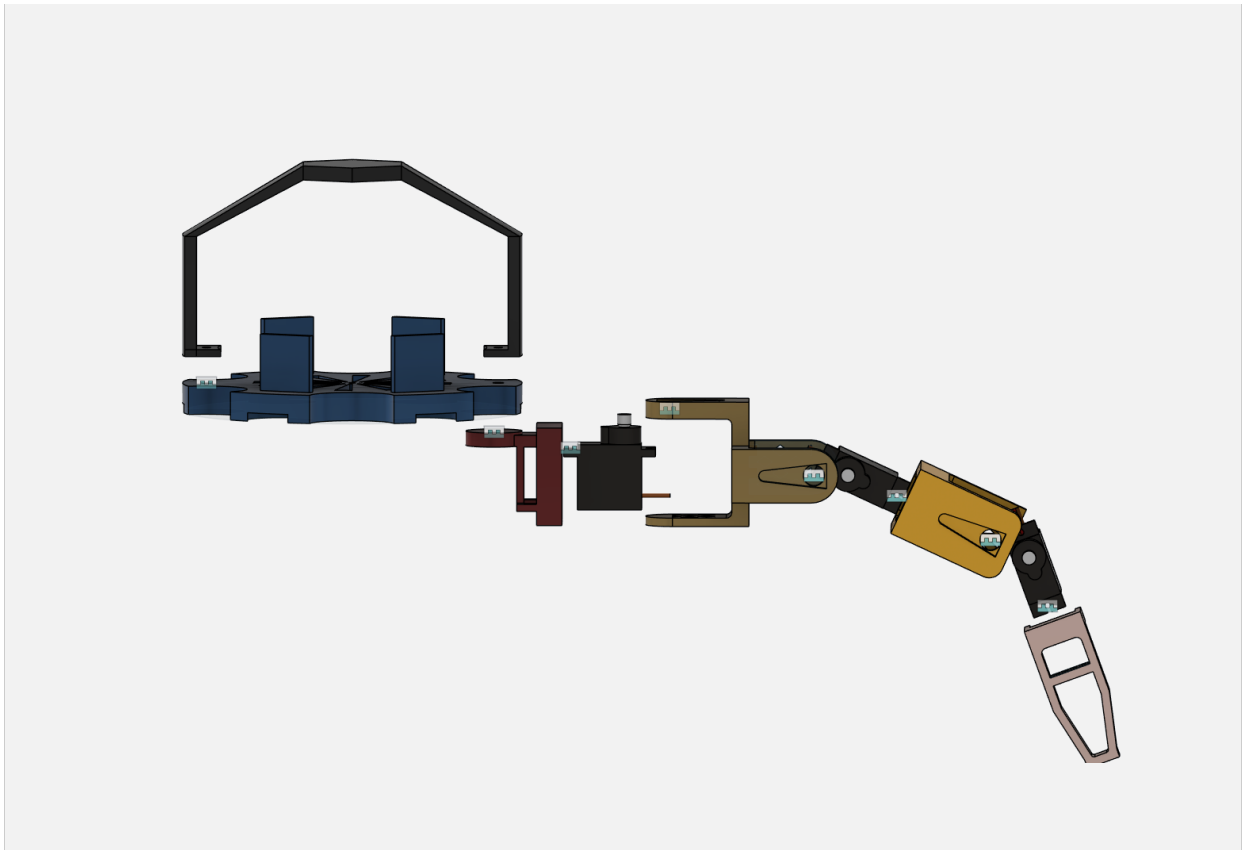
- 34. Wire battery T-plug (female) to power switch using 22AWG wire.
- 35. Wire switch to both buck converter inputs.
- 36. Set both buck converters to 5V output using a multimeter or their onboard display before connecting any load.
- 37. Wire XH-M404 output to both PCA9685 V+ terminals, with one 1000 μ F capacitor at each.
- 38. Wire LM2596 output to Nano VIN.
- 39. Wire Nano 5V/GND to both PCA9685 VCC/GND and to HC-SR04 VCC/GND.
- 40. Wire Nano A4/A5 to both PCA9685 SDA/SCL.

41. Wire Nano D7/D8 to HC-SR04 TRIG/ECHO.
42. Connect all 18 servos to their assigned PCA9685 channels per the Channel Map (Section 7.2).

9. Exploded View & Final Assembly

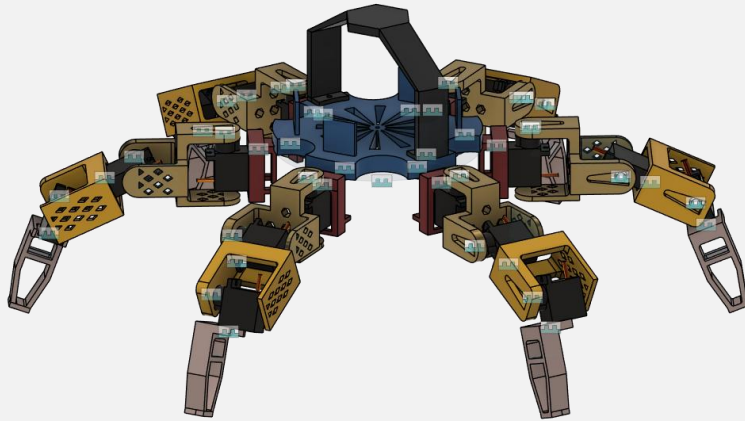
9.1 Exploded View

All major components shown separated along their assembly axis, illustrating how each part stacks relative to the others.



9.2 Final Assembled Model

Fully joined CAD assembly, representing the completed robot prior to wiring.



10. Testing & Calibration Checklist

- ☐ All 18 servos respond individually via PCA9685 (no legs attached)
- ☐ Buck converter outputs verified at 5V before connecting any board
- ☐ Robot stands stably on all 6 legs at home position
- ☐ HC-SR04 readings stable and accurate (compare against tape measure at 10/30/50cm)
- ☐ Single wave gait cycle completes without any leg dragging or losing balance
- ☐ Obstacle avoidance correctly triggers stop + turn sequence
- ☐ Battery voltage alarm triggers correctly at set threshold
- ☐ No wire or connector becomes warm to the touch during 5+ minutes of continuous walking

11. Troubleshooting

Symptom	Likely Cause	Fix
Servo grinds/buzzes	Loose/intermittent signal connection	Solder signal wire instead of friction-fit jumper

Print cracks along Z-axis at wall transition	Sharp corner stress concentration + thin wall under overhang	Add fillet (R3–R5mm) at the transition, increase local wall thickness
Servo wobbles in single-screw mount	Clone servo flange not flat / single fastening point	Add epoxy on contact faces, or wrap joint with wire + tape as a field fix
Arduino resets/browns out during gait	Servo current spike on shared power rail	Verify dual buck converter separation, confirm 1000 μ F capacitors are installed at each PCA9685
IDE can't upload, "programmer not responding"	Wrong bootloader selected for clone Nano	Try both Old Bootloader and standard ATmega328P options in Tools → Processor
False obstacle triggers during gait	Ultrasonic beam clipping a swinging leg	Mount HC-SR04 higher, or add consecutive-reading filtering in firmware
Robot tips over while walking	Home position not centered, or step height/length too aggressive for current gait	Re-tune home angles per leg; reduce STEP_COXA_SWING and LIFT_FEMUR_DEG

12. Roadmap

- v1.0 (current) — wave gait locomotion, ultrasonic obstacle avoidance
- v2.0 — reinforcement learning locomotion trained in Unreal Engine simulation
- v3.0 — onboard camera + visual navigation
- v4.0 — sim-to-real policy transfer to physical platform

Appendix A — Full Firmware Source

Complete Arduino sketch (HexEcho_firmware.ino). The authoritative copy lives in /firmware in the repository — this listing is for offline reference.

```
/*
 * =====
 * HexEcho Hexapod Firmware - v2 (with interpolation engine)
 * =====
 * Hardware:
 *   Arduino Nano, 2x PCA9685 (0x40 / 0x41), HC-SR04, MG90S + SG90
 *
 * Gait: Wave gait (1 leg at a time) - most stable
 * Motion: All servo moves go through the interpolation engine.
 *         Servos never snap - they glide from current to target.
 *
 * -----
 * CHANNEL MAP
 * -----
 * PCA0 (0x40) - LEFT SIDE legs 0,1,2 (Front / Mid / Back)
 *   Ch 0,1,2 = Leg0 Coxa, Femur, Tibia
 *   Ch 3,4,5 = Leg1 Coxa, Femur, Tibia
 *   Ch 6,7,8 = Leg2 Coxa, Femur, Tibia
 *
 * PCA1 (0x41) - RIGHT SIDE legs 3,4,5 (Front / Mid / Back)
 *   Ch 0,1,2 = Leg3 Coxa, Femur, Tibia
 *   Ch 3,4,5 = Leg4 Coxa, Femur, Tibia
 *   Ch 6,7,8 = Leg5 Coxa, Femur, Tibia
 *
 * Top-view layout (front = top):
 *   [FRONT]
 *   L0      R3
 *   L1      R4
 *   L2      R5
 *   [BACK]
 *
 * -----
 * HOME POSITION SUMMARY (what the code commands on boot)
 * -----
 *   Coxa = 90° (straight sideways)
 *   Femur = 60° (tilted slightly forward-down for a standing pose)
 *   Tibia = 110° (angled to plant foot flat on ground)
 *
 * Mount each servo FIRST to these angles - see the mounting guide PDF.
 * =====
 */

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

// =====
// USER-CONFIGURABLE - edit to match your hardware
// =====

// Leg segment lengths in mm (only used for reference / future IK)
#define COXA_LEN_MM 60.0f
#define FEMUR_LEN_MM 50.0f
#define TIBIA_LEN_MM 70.0f

// Home stance angles
#define COXA_HOME_DEG 90
#define FEMUR_HOME_DEG 60
#define TIBIA_HOME_DEG 110

// Gait geometry
#define STEP_COXA_SWING 25 // degrees coxa travels per step
```

```

#define LIFT_FEMUR_DEG    20    // degrees femur rises during swing
#define TURN_COXA_OFFSET  22    // extra coxa offset used during rotation

// Interpolation speed – lower = slower / smoother
// INTERP_STEP_DEG: how many degrees to advance per tick
// INTERP_TICK_MS : ms between each tick
// At 1 deg / 15 ms → a 30-degree move takes ~450 ms
#define INTERP_STEP_DEG    1    // degrees per tick (keep at 1 for smoothest)
#define INTERP_TICK_MS    12    // ms per tick

// Pauses between gait phases (on top of interpolation time)
#define PHASE_SETTLE_MS    120    // brief settle after each phase completes
#define STEP_SETTLE_MS    300    // pause after each leg finishes its step
#define CYCLE_SETTLE_MS    700    // pause after a full 6-leg cycle – SAFE POWER-OFF WINDOW

// HC-SR04 pins
#define TRIG_PIN    7
#define ECHO_PIN    8

// Obstacle distance threshold
#define OBSTACLE_CM    20

// PCA9685 pulse range – tune if servos don't reach 0° or 180°
#define PWM_MIN    150    // pulse count at 0°
#define PWM_MAX    600    // pulse count at 180°

// =====
// PCA9685 INSTANCES
// =====
Adafruit_PWMServoDriver pca0 = Adafruit_PWMServoDriver(0x40);
Adafruit_PWMServoDriver pca1 = Adafruit_PWMServoDriver(0x41);

// =====
// LEG STRUCTURE
// currentDeg[] = live angles (updated every interpolation tick)
// targetDeg[] = where we want to go
// mirrorCoxa = true for right-side legs (inverts coxa direction)
// =====
struct Leg {
    uint8_t board;           // 0 = pca0, 1 = pca1
    uint8_t ch[3];           // channel indices: 0=coxa 1=femur 2=tibia
    float currentDeg[3];     // current servo angle (float for smooth interp)
    int targetDeg[3];        // target angle
    bool mirrorCoxa;         // true = invert coxa for right-side legs
};

Leg legs[6];

// =====
// LOW-LEVEL SERVO WRITE
// =====

uint16_t degToPulse(float deg) {
    deg = constrain(deg, 0.0f, 180.0f);
    return (uint16_t)map((long)(deg * 10), 0, 1800, PWM_MIN, PWM_MAX);
}

void writeServoRaw(uint8_t board, uint8_t channel, float deg) {
    uint16_t pulse = degToPulse(deg);
    if (board == 0) pca0.setPWM(channel, 0, pulse);
    else            pca1.setPWM(channel, 0, pulse);
}

// Flush current angles to hardware for one leg
void flushLeg(uint8_t li) {
    Leg& L = legs[li];
    float coxaDeg = L.mirrorCoxa ? (180.0f - L.currentDeg[0]) : L.currentDeg[0];

```

```

writeServoRaw(L.board, L.ch[0], coxaDeg);
writeServoRaw(L.board, L.ch[1], L.currentDeg[1]);
writeServoRaw(L.board, L.ch[2], L.currentDeg[2]);
}

// =====
// INTERPOLATION ENGINE
// Moves all 6 legs from their currentDeg toward their targetDeg
// one INTERP_STEP_DEG tick at a time.
// Returns only when all legs have reached their targets.
// =====
void interpolateAllLegs() {
    bool moving = true;
    while (moving) {
        moving = false;
        for (uint8_t li = 0; li < 6; li++) {
            for (uint8_t j = 0; j < 3; j++) {
                float diff = legs[li].targetDeg[j] - legs[li].currentDeg[j];
                if (abs(diff) > 0.5f) {
                    moving = true;
                    float step = constrain(diff, -(float)INTERP_STEP_DEG, (float)INTERP_STEP_DEG);
                    legs[li].currentDeg[j] += step;
                } else {
                    legs[li].currentDeg[j] = (float)legs[li].targetDeg[j];
                }
            }
            flushLeg(li);
        }
        if (moving) delay(INTERP_TICK_MS);
    }
    // One final flush after motion completes – ensures all servos are exactly on target
    for (uint8_t li = 0; li < 6; li++) flushLeg(li);
}

// =====
// HOLD DELAY
// Replaces all bare delay() calls during pauses.
// Re-flushes every servo every HOLD_REFRESH_MS during the wait
// so the PCA9685 keeps asserting PWM and servos don't go limp.
// Also catches clone PCA9685 boards that occasionally lose state.
// =====
#define HOLD_REFRESH_MS 20 // re-send PWM every 20ms during holds

void holdDelay(uint32_t ms) {
    uint32_t start = millis();
    while (millis() - start < ms) {
        for (uint8_t li = 0; li < 6; li++) flushLeg(li);
        delay(HOLD_REFRESH_MS);
    }
}

// Convenience: set target for one leg's joint (0=coxa,1=femur,2=tibia)
void setTarget(uint8_t li, uint8_t joint, int deg) {
    legs[li].targetDeg[joint] = constrain(deg, 0, 180);
}

// Set all three joints of one leg at once
void setLegTarget(uint8_t li, int coxaDeg, int femurDeg, int tibiaDeg) {
    legs[li].targetDeg[0] = constrain(coxaDeg, 0, 180);
    legs[li].targetDeg[1] = constrain(femurDeg, 0, 180);
    legs[li].targetDeg[2] = constrain(tibiaDeg, 0, 180);
}

// =====
// LEG TABLE INIT
// =====
void initLegsTable() {
    // Helper lambda-style inline – sets a leg entry
    // Left legs (0,1,2): mirrorCoxa = false

```

```

// Right legs (3,4,5): mirrorCoxa = true
struct LegInit { uint8_t board; uint8_t c, f, t; bool mirror; };
const LegInit cfg[6] = {
    {0, 0, 1, 2, false}, // Leg0 L-Front
    {0, 3, 4, 5, false}, // Leg1 L-Mid
    {0, 6, 7, 8, false}, // Leg2 L-Back
    {1, 0, 1, 2, true }, // Leg3 R-Front
    {1, 3, 4, 5, true }, // Leg4 R-Mid
    {1, 6, 7, 8, true }, // Leg5 R-Back
};
for (uint8_t i = 0; i < 6; i++) {
    legs[i].board      = cfg[i].board;
    legs[i].ch[0]      = cfg[i].c;
    legs[i].ch[1]      = cfg[i].f;
    legs[i].ch[2]      = cfg[i].t;
    legs[i].mirrorCoxa = cfg[i].mirror;
    // Start current and target both at home
    legs[i].currentDeg[0] = COXA_HOME_DEG;
    legs[i].currentDeg[1] = FEMUR_HOME_DEG;
    legs[i].currentDeg[2] = TIBIA_HOME_DEG;
    legs[i].targetDeg[0]  = COXA_HOME_DEG;
    legs[i].targetDeg[1]  = FEMUR_HOME_DEG;
    legs[i].targetDeg[2]  = TIBIA_HOME_DEG;
}
}

// =====
// HOME POSITION
// Moves legs to home one at a time so the bot doesn't lurch.
// =====
void goHome() {
    Serial.println(F("[HOME] Moving to home position..."));
    for (uint8_t i = 0; i < 6; i++) {
        setLegTarget(i, COXA_HOME_DEG, FEMUR_HOME_DEG, TIBIA_HOME_DEG);
        interpolateAllLegs();
        holdDelay(PHASE_SETTLE_MS);
    }
    holdDelay(CYCLE_SETTLE_MS);
    Serial.println(F("[HOME] Done."));
}

// =====
// SINGLE LEG STEP (lift → swing → plant)
// direction : +1 = forward, -1 = backward
// coxaExtra : extra coxa offset for turning
//
// While one leg moves, the others stay put (wave gait).
// =====
void stepLeg(uint8_t li, int direction, int coxaExtra) {
    int targetCoxa = COXA_HOME_DEG + (direction * STEP_COXA_SWING) + coxaExtra;
    targetCoxa = constrain(targetCoxa, 45, 135);

    // --- Phase 1: Lift ---
    setLegTarget(li, COXA_HOME_DEG,
                  FEMUR_HOME_DEG - LIFT_FEMUR_DEG,
                  TIBIA_HOME_DEG);
    interpolateAllLegs();
    holdDelay(PHASE_SETTLE_MS);

    // --- Phase 2: Swing ---
    setLegTarget(li, targetCoxa,
                  FEMUR_HOME_DEG - LIFT_FEMUR_DEG,
                  TIBIA_HOME_DEG);
    interpolateAllLegs();
    holdDelay(PHASE_SETTLE_MS);

    // --- Phase 3: Plant ---
    setLegTarget(li, targetCoxa,
                  FEMUR_HOME_DEG,
                  TIBIA_HOME_DEG);

```

```

    interpolateAllLegs();
    holdDelay(STEP_SETTLE_MS); // *** SAFE POWER-OFF POINT – leg planted, stable ***
}

// =====
// RESET ALL COXAS TO HOME (after a full gait cycle)
// =====
void resetAllCoxas() {
    for (uint8_t i = 0; i < 6; i++) {
        setTarget(i, 0, COXA_HOME_DEG);
    }
    interpolateAllLegs();
    holdDelay(PHASE_SETTLE_MS);
}

// =====
// WAVE GAIT – FORWARD / BACKWARD
// Sequence: L-Front → R-Front → L-Mid → R-Mid → L-Back → R-Back
// (alternates sides to keep weight balanced)
// =====
const uint8_t WAVE_ORDER[6] = {0, 3, 1, 4, 2, 5};

void waveGait(int direction) {
    for (uint8_t i = 0; i < 6; i++) {
        stepLeg(WAVE_ORDER[i], direction, 0);
    }
    resetAllCoxas();
    holdDelay(CYCLE_SETTLE_MS); // *** SAFE POWER-OFF WINDOW ***
}

void stepForward() { Serial.println(F("[GAIT] Forward")); waveGait(+1); }
void stepBackward() { Serial.println(F("[GAIT] Backward")); waveGait(-1); }

// =====
// ROTATION (turn in place)
// rotDir: +1 = left, -1 = right
//
// Left turn: left legs push back, right legs push forward
// Right turn: left legs push forward, right legs push back
// =====
void rotateStep(int rotDir) {
    for (uint8_t i = 0; i < 6; i++) {
        uint8_t li = WAVE_ORDER[i];
        // Legs 0,1,2 = left → opposite direction to rotDir
        // Legs 3,4,5 = right → same direction as rotDir
        int legDir = (li < 3) ? (-rotDir) : (rotDir);
        int coxaExtra = TURN_COXA_OFFSET * legDir;
        stepLeg(li, legDir, coxaExtra);
    }
    resetAllCoxas();
    holdDelay(CYCLE_SETTLE_MS);
}

void turnLeft() { Serial.println(F("[TURN] Left")); rotateStep(+1); }
void turnRight() { Serial.println(F("[TURN] Right")); rotateStep(-1); }

// =====
// HC-SR04 SENSOR
// =====
float getDistanceCm() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(4);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long dur = pulseIn(ECHO_PIN, HIGH, 26000UL); // timeout ~4.5m
    if (dur == 0) return 999.0f;
    return (dur * 0.0343f) / 2.0f;
}

```

```

bool obstacleAhead() {
    float d = getDistanceCm();
    Serial.print(F("[SENSOR] "));
    Serial.print(d, 1);
    Serial.println(F(" cm"));
    return (d < OBSTACLE_CM);
}

// =====
// OBSTACLE AVOIDANCE
//
// Logic:
// 1. Clear ahead → step forward
// 2. Blocked → try LEFT
// 3. Still blocked → turn RIGHT x2 (undo left, then face right)
// 4. Still blocked → turn RIGHT once more (~facing backward)
// 5. Still blocked → go home, halt (safe power-off)
// =====
void obstacleAvoidanceLoop() {
    Serial.println(F("--- Checking ahead ---"));

    if (!obstacleAhead()) {
        stepForward();
        return;
    }

    Serial.println(F("[NAV] Obstacle! Trying LEFT..."));
    turnLeft();
    if (!obstacleAhead()) { stepForward(); return; }

    Serial.println(F("[NAV] Still blocked. Turning RIGHT x2..."));
    turnRight(); // undo left → back to original heading
    turnRight(); // now facing right
    if (!obstacleAhead()) { stepForward(); return; }

    Serial.println(F("[NAV] Still blocked. Turning RIGHT once more..."));
    turnRight(); // now facing ~180° from original
    if (!obstacleAhead()) { stepForward(); return; }

    Serial.println(F("[NAV] ALL DIRECTIONS BLOCKED. Returning home and halting.));
    goHome();
    while (true) holdDelay(500); // keeps servo PWM alive at home position
}

// =====
// SETUP
// =====
void setup() {
    Serial.begin(9600);
    Serial.println(F("=== HexEcho booting ==="));

    Wire.begin();

    pca0.begin(); pca0.setPWMPFreq(50);
    pca1.begin(); pca1.setPWMPFreq(50);
    delay(200);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    initLegsTable();

    // Flush initial angles to hardware immediately (no snap – already at home)
    for (uint8_t i = 0; i < 6; i++) flushLeg(i);
    holdDelay(300);
}

```

```
// Smooth move to home (in case servos powered up in unknown position)
goHome();

Serial.println(F("=== Ready ==="));
holdDelay(1000);
}

// =====
//  MAIN LOOP
//  =====
void loop() {
  obstacleAvoidanceLoop();
  holdDelay(CYCLE_SETTLE_MS);
}
```

Appendix B — Credits, License & Revision History

Credits

Designed, built, and documented as an independent course project — mechanical design, electronics, and firmware all developed from first principles.

- Mechanical design — Autodesk Fusion 360
- Circuit schematic — EasyEDA
- Firmware — Arduino IDE, Adafruit_PWMServoDriver library
- Chassis fabrication — PETG FDM 3D printing

License

This project is released under the MIT License. See LICENSE in the repository root for full terms.

Revision History

Version	Date	Changes
v1.0	2026	Initial build — wave gait locomotion, ultrasonic obstacle avoidance